SHELLSPEC Docs - English

Table of contents

- Architecture 3
- Cli 4
- Directory_structure 7
- Docker 9
- References 12
- Reporter 30
- Shells 32
- Test 39

Architecture

```
shellspec command                                specfile execution
+--------------------------------+               +--------------------------------+
| 1. shellspec                   |               | The translated specfile is     |
|   Parsing options.             |               | just a plain shell script      |
|                                |               | that include the core scripts  |
| 2. shellspec-runner.sh         |               | of shellspec.                  |
|   Execute executor and         |               |                                |
|   reporter.                    |               | It is executed in a separate   |
|     |                          | translated    | process from shellspec         |
|     v                          | specfile      | command.                       |
|   3. shellspec-executor.sh ---|------------->|                                |
|     Execute translator and     |               | Do not use external command,   |
|     translated specfile.       |               | subshell, pipe, and command    |
|       |                        |               | substituion from core scripts  |
|       v                        |               | as much as possibe for         |
|     4. shellspec-translate.sh  |               | performance and portability.   |
|       Translate specfile.      | reporting     |                                |
|                                | protocol      | Currently, only command-based  |
| 5. shellspec-reporter.sh <----|-------------| mocks use external commands.   |
|   Reporting.                   |               |                                |
|                                |               | This rule applies only to      |
| Can be used POSIX compliant    |               | core scripts. You can use      |
| commands, but make as less     |               | external commands freely in    |
| as possible.                   |               | your test code.                |
+--------------------------------+               +--------------------------------+
```

Go to TOC

ShellSpec CLI

- Initialize your project (`--init`)
- Specify the shell to run (`--shell`)
- Quick execution (`--quick` , `--repair` , `--next`)
- Parallel execution (`--jobs`)
- Random execution (`--random`)
- Fail fast (`--fail-fast`)
- Trace (`--xtrace` , `--xtrace-only`)
- Sandbox mode (`--sandbox`)
- Ranges (`:LINENO` , `:@ID`) / Filters (`--example`) / Focus (`--focus`)
- Reporter (`--format`) / Generator (`--output`)
- Profiler (`--profile`)
- Run tests in Docker container (`--docker`)
- Task runner (`--task`)

Initialize your project (`--init`)

Run `shellspec --init` initializes the current directory for ShellSpec. It creates `.shellspec` and `spec/spec_helper.sh`

Specify the shell to run (`--shell`)

Specify the shell to run with `--shell` option. ShellSpec ignores shebang and runs the shell script in the specified shell. The default is the shell running the `shellspec` command (usually `/bin/sh`).

Quick execution (`--quick` , `--repair` , `--next`)

Quick execution is a feature for rapid development and failure fixing.

When you run `shellspec` with `--quick` option first, Quick mode is automatically enabled. When Quick mode enabled, The file `.shellspec-quick.log` generated on the project root directory. If you want to disable Quick mode, delete `.shellspec-quick.log` .

When Quick mode enabled, the results of running examples are logged to `.shellspec-quick.log` on the project root directory (even if `--quick` option is not specified).

Use `--quick` option is for rapid development. When `--quick` option specified, It runs examples that not-passed (failure and temporary pending) the last time they ran. If there are no examples that did not pass, It runs all examples. It is designed to be added to `$HOME/.shellspec` instead of being specified each runs.

Use `--repair` and `--next` option is for rapid failure fixing. It runs failed examples only (not includes temporary pending).

Parallel execution (`--jobs`)

You can use parallel execution for fast test with `--jobs` option. Parallel jobs are executed per specfile. So it is necessary to separate the specfile for effective parallel execution.

Random execution (`--random`)

You can randomize the execution order to detect troubles due to the test execution order. If `SEED` is specified, the execution order is deterministic.

Fail fast (`--fail-fast`)

You can stop on the first (N times) failures with `--fail-fast` option.

NOTE: The reporter that count the number of failures and specfile execution are processed in parallel. Therefore, the specfile execution may precede the location where it stopped due to a failure.

Trace (`--xtrace`, `--xtrace-only`)

You can trace evaluation with `--xtrace` or `--xtrace-only` option.

If `BASH_XTRACEFD` is implemented in the shell, you can run tests and traces at the same time. Otherwise, run tracing only. The output format can be set with the variable `PS4`.

NOTE: `BASH_XTRACEFD` only available *bash version >= 4.1* or *busybox (ash) version >= 1.28.0*.

Sandbox mode (`--sandbox`)

Force the use of the mock instead of the actual command. This option makes the `PATH` environment variable empty (except `spec/support/bin`) and `readonly`.

Support commands help to call the actual command in sandbox mode.

NOTE: This is not a security feature and does not provide complete isolation. For example, if specified with an absolute path, the actual command will be executed. If you need strict isolation, use Docker or similar technology.

Ranges (`:LINENO`, `:@ID`) / Filters (`--example`) / Focus (`--focus`)

You can run specific example(s) or example group(s) only.

It can be specified by line number (`a_spec.sh:10:20`), example id (`a_spec.sh:@1-5:@1-6`), example name (`--example` option), tag (`--tag` option) and focus (`--focus` option).

To focus, prepend `f` to groups / examples in specfiles (e.g. `Describe` -> `fDescribe`, `It` -> `fIt`) and run with `--focus` option.

Reporter (`--format`) / Generator (`--output`)

You can specify one reporter (output to stdout) and multiple generators (output to a file). Currently builtin formatters are `progress`, `documentation`, `tap`, `junit`, `failures`, `null`, `debug`.

NOTE: Custom formatter is supported (but not documented yet, sorry).

Profiler (`--profile`)

When the `--profile` option is specified, the profiler is enabled and lists the slow examples.

Run tests in Docker container (`--docker`)

NOTE: This is an experimental feature and may be changed/removed in the future.

When the `--docker DOCKER-IMAGE` option is specified, run tests using the specified Docker image.

If you specify only the tag that starts with `:` as DOCKER-IMAGE (e.g. `--docker :debian10`), Use ShellSpec official runtime image (`shellspec/runtime`). The ShellSpec official runtime image contains supported shells.

See available tags: https://hub.docker.com/r/shellspec/runtime/tags

Task runner (`--task`)

You can run the task with `--task` option.

Go to TOC

Project directory structure

Example directory structure and options

Separate a separate directory for each utility and test it with `test_spec.sh`. The execution directory for testing is where the specfile is located.

```
<PROJECT-ROOT>
├─ .shellspec
|
├─ script1/
|    ├─ script1.sh
|    └─ test_spec.sh
|
├─ script2/
|    ├─ script2.sh
|    └─ test_spec.sh
|
├─ spec/
|    ├─ spec_helper.sh
|    |         :
```

```
# .shellspec
--default-path "**/test_spec.sh"
--execdir @specfile
```

Separate a separate directory for each utility and test it with `spec/*_spec.sh`. The test execution directory is where the script is located.

```
<PROJECT-ROOT>
├─ .shellspec
|
├─ script1/
|    ├─ .shellspec-basedir
|    ├─ bin/
|    |    ├─ script1.sh
|    |    |     :
|    └─ spec/
|         ├─ script1_spec.sh
|         |     :
|
├─ script2/
|    ├─ .shellspec-basedir
|    ├─ bin/
|    |    ├─ script2.sh
|    |    |     :
|    └─ spec/
|         ├─ script2_spec.sh
|         |     :
|
├─ spec/
|    ├─ spec_helper.sh
|    ├─ support/
|    |         :
```

```
# .shellspec
--default-path "**/spec"
--execdir @basedir/bin`
```

Go to TOC

How to use ShellSpec with Docker

- Official docker images
- Using ShellSpec docker image
  - 1. Run ShellSpec and your specfiles within container
  - 2. Run simple with helper script and extra hooks
    - Hooks
      - .shellspec-docker/initrc
      - .shellspec-docker/pre-test
      - .shellspec-docker/post-test
  - 3. Using ShellSpec image as parent image
  - 4. Include ShellSpec into another image
- Appendix
  - How to build official ShellSpec docker image yourself

Official docker images

There are official images on the Docker Hub.

Name	Linux	Included	Size
shellspec/shellspec	Alpine	busybox (ash)	2.88 MB
shellspec/shellspec:kcov	Alpine	busybox (ash), bash, kcov	14.3 MB
shellspec/shellspec-debian	Debian	dash, bash	50.5 MB
shellspec/shellspec-debian:kcov	Debian	dash, bash, kcov	70.9 MB
shellspec/shellspec-scratch	None	none (shellspec only)	76.6 kB

- Version specified images are also available (VERSION: 0.21.0 and above)
  - `shellspec/shellspec[-VARIANT]:<VERSION | master>[-kcov]`

Using ShellSpec docker image

1. Run ShellSpec and your specfiles within container

```
# Run docker command on the project root
$ docker run -it --rm -v "$PWD:/src" shellspec/shellspec

# Display help
$ docker run -it --rm -v "$PWD:/src" shellspec/shellspec --help
```

```
# Run with kcov (requires kcov supported image)
$ docker run -it --rm -u $(id -u):$(id -g) \
    -v "$PWD:/src" shellspec/shellspec:kcov --kcov

# For users using Docker Desktop for Windows within WSL 1
$ docker run -it --rm -v "$(wslpath -wa .):/src" shellspec/shellspec
```

2. Run simple with helper script and extra hooks

Use contrib/shellspec-docker helper script.

```
# Specify the Docker image to use (default: shellspec/shellspec)
$ export SHELLSPEC_DOCKER=shellspec/shellspec

# Run helper script on the project root
$ shellspec-docker

# Display help
$ shellspec-docker --help

# Run with kcov (requires kcov supported image)
$ shellspec-docker --kcov

# Enter the Docker container
$ shellspec-docker -

# Execute command with in the Docker container
$ shellspec-docker - hostname
```

If you want to run manually.

```
$ docker run -it --rm --entrypoint=/shellspec-docker \
    -u $(id -u):$(id -g) -v "$PWD:/src" shellspec/shellspec

# For users using Docker Desktop for Windows within WSL 1
$ docker run -it --rm --entrypoint=/shellspec-docker \
    -u $(id -u):$(id -g) -v "$(wslpath -wa .):/src" shellspec/shellspec
```

Hooks

.shellspec-docker/initrc

This file should be a shell script. You can override docker_run() to changes options, pass environment variables, etc.

.shellspec-docker/pre-test

Invoked before execute shellspec inside of the docker container.

.shellspec-docker/post-test

Invoked after executed shellspec inside of the docker container.

3. Using ShellSpec image as parent image

Example

```
# Dockerfile
FROM shellspec/shellspec
RUN apk add --no-cache add-your-required-packages
COPY ./ /src
```

```
# Build and run at your project root
$ docker build -t your-project-name .
$ docker run -it your-project-name
```

4. Include ShellSpec into another image

Example

```
# Dockerfile
FROM buildpack-deps
RUN apt-get update && apt-get install -y add-your-required-packages
COPY --from=shellspec/shellspec-scratch /opt/shellspec /opt/shellspec
ENV PATH /opt/shellspec/:$PATH
WORKDIR /src
ENTRYPOINT [ "shellspec" ]
COPY ./ /src
```

```
# Build and run at your project root
$ docker build -t your-project-name .
$ docker run -it your-project-name
```

Appendix

How to build official ShellSpec docker image yourself

Example

```
contrib/build.sh .dockerhub/Dockerfile         shellspec
contrib/build.sh .dockerhub/Dockerfile         shellspec kcov
contrib/build.sh .dockerhub/Dockerfile.debian  shellspec-debian
contrib/build.sh .dockerhub/Dockerfile.debian  shellspec-debian kcov
contrib/build.sh .dockerhub/Dockerfile.scratch shellspec-scratch
```

Go to TOC

References

- Basic structure
  - Example group
    - `ExampleGroup` / `Describe` / `Context`
  - Example
    - `Example` / `It` / `Specify`
  - Evaluation
    - `When call`
    - `When run`
      - about calling shell function with run
    - `When run command`
    - `When run script`
    - `When run source`
    - Comparison
  - Expectation
    - `The` ... `should (not)`
    - `Assert`
  - Subjects
    - `stdout` (`output`) subject
    - `stderr` (`error`) subject
    - `status` subject
    - `line` subject
    - `word` subject
    - `path` / `file` / `directory` subject
    - `function` subject
    - `value` subject
    - `variable` subject
  - Modifiers
    - `line` modifier
    - `lines` modifier
    - `word` modifier
    - `length` modifier
    - `contents` modifier
    - `result` modifier
  - Matchers
    - `satisfy` matcher
    - stat matchers
      - `be exist` matcher
      - `be file` matcher
      - `be directory` matcher

        - `be empty file` matcher
        - `be empty directory` matcher
        - `be symlink` matcher
        - `be pipe` matcher
        - `be socket` matcher
        - `be readable` matcher
        - `be writable` matcher
        - `be executable` matcher
        - `be block_device` matcher
        - `be character_device` matcher
        - `have setgid` matcher
        - `have setuid` matcher
    - status matchers
        - `be success` matcher
        - `be failure` matcher
    - string matchers
        - `equal` matcher
        - `start with` matcher
        - `end with` matcher
        - `include` matcher
        - `match pattern` matcher
    - `successful` matcher
    - valid matchers
    - variable matchers
        - `be defined` matcher
        - `be undefined` matcher
        - `be present` matcher
        - `be blank` matcher
        - `be exported` matcher
        - `be readonly` matcher
- Helper
    - Skip / Pending
        - `Skip`
        - `Skip if`
        - `Pending`
        - `Todo`
    - Data
        - `Data[:raw]`
        - `Data:expand`
        - `Data <FUNCTION>`
        - `Data "<STRING>"`
        - `Data < "<FILE>"`

- Parameters
  - `Parameters[:block]`
  - `Parameters:value`
  - `Parameters:matrix`
  - `Parameters:dynamic`
- Others
  - `Include`
  - `Path` / `File` / `Dir`
  - `Intercept`
  - `Set`
  - `Dump`
- Hooks
  - `Before` / `After`
  - `BeforeAll` / `AfterAll`
  - `BeforeCall` / `AfterCall`
  - `BeforeRun` / `AfterRun`
- Directive
  - `%const` (`%`)
  - `%text`
  - `%puts` (`%-`) / `%putsn` (`%=`)
  - %preserve
  - `%logger`
- Special environment Variables

Basic structure

You can write a structured *Example* by using the DSL shown below:

Example group

DSL	Description
ExampleGroup ... End	Define an example group.
Describe ... End	Synonym for `ExampleGroup`.
Context ... End	Synonym for `ExampleGroup`.

`ExampleGroup` / `Describe` / `Context`

Example groups are nestable.

Example

DSL	Description
Example ... End	Define an example.

DSL	Description
It ... End	Synonym for `Example`.
Specify ... End	Synonym for `Example`.

`Example` / `It` / `Specify`

Evaluation

The line beginning with `When` is the evaluation.

Evaluation	Description
When call	Call shell function without subshell.
When run	Run shell function or external command within a subshell.
When run command	Run external command (including non-shell scripts).
When run script	Run shell script by new process of the current shell.
When run source	Run shell script in the current shell by `.` command (aka `source`).

`When call`

```
When call <FUNCTION> [ARGUMENTS...]
```

This is primarily designed for shell function calls. It is the recommended evaluation as a unit test. It does not use a subshell, therefore it is the fastest evaluation variant and you can assert variables.

`When run`

```
When run <FUNCTION | COMMAND> [ARGUMENTS...]
```

This is primarily designed for external command calls. The external command does not have to be a shell script. Even shell scripts are executed as external commands according to the shebang, so they are not covered by the coverage.

about calling shell function with run

If a shell function is specified, it will be executed in a subshell. The slight advantage of executing shell functions with `run` is that you can trap errors with `set -e`. Unlike `call`, it does not cause an error, so you can assert the exit status.

Also, because of the execution in the subshell, the variables which change values in the function are restored once `run` finishes. This is often a disadvantage, but tests of ShellSpec itself intentionally use `run` because changing internal variables confuses ShellSpec's behavior.

If you want to assert variables with `run`, use the `%preserve` directive in function called by `AfterRun` hook. It can preserve variables even if `run` exits the subshell.

`When run command`

```
When run command <COMMAND> [ARGUMENTS...]
```

Run an external command explicitly. The external command does not have to be a shell script. Even shell scripts are executed as external commands according to the shebang, so they are not covered by the coverage.

`When run script`

```
When run script <SCRIPT> [ARGUMENTS...]
```

Run the external shell script in the same shell as the currently running shell.

`When run source`

```
When run source <SCRIPT> [ARGUMENTS...]
```

This is similar to `run script`, but simulates the running of shell scripts using the `.` command instead of running directly.

The advantage over `run script` is that you can use `Intercept` to intercept at any point in the external shell script. This is useful for preparation testing and mocking with shell functions.

Comparison

	`call`	**`run`**	**`run command`**	**`run script`**	**`run source`**
Run in subshell	No	Yes	Yes	Yes	Yes
Target	function	function / command	command	shell script	shell script
Stop with `set -e`	No	Yes	-	Yes	Yes
Catch `exit`	No	Yes	-	Yes	Yes
Expectation Hooks	BeforeCall / AfterCall	BeforeRun / AfterRun	BeforeRun / AfterRun	BeforeRun / AfterRun	BeforeRun / AfterRun
Intercept	No	No	-	No	Yes
Coverage	Yes	Yes (function only)	No	Yes	Yes

Expectation

`The` ... `should (not)`

The line beginning with `The` is the evaluation. The *subject* or the *modifier* follows after `The`. And last is the *matcher*.

```
The [MODIFIER of...] <SUBJECT> should <MATCHER>
The [MODIFIER of...] <SUBJECT> should not <MATCHER>
```

`Assert`

```
Assert <FUNCTION> [ARGUMENTS...]
```

Subjects

Subject	Description
stdout / output	Use the stdout of *Evaluation* as subject.
line	Same as `line NUMBER of stdout`.
word	Same as `word NUMBER of stdout`.
stderr / error	Use the stderr of *Evaluation* as subject.
status	Use the status of *Evaluation* as subject.
path / file / directory	Use the alias resolved path as the subject.
value	Use the value as the subject.
function	Use the function name as the subject.
variable	Use the value of the variable as the subject.

`stdout` (`output`) subject

```
The stdout should equal "foo"
The output should equal "foo"
```

`stderr` (`error`) subject

```
The stderr should equal "foo"
The error should equal "foo"
```

`status` subject

```
The status should be success
```

`line` subject

When combined with line, `stdout` can be omitted.

```
The line 1 of stdout should equal foo
The line 1 should equal foo # stdout omitted
```

`word` subject

When combined with word, `stdout` can be omitted.

```
The word 1 of stdout should equal foo
The word 1 should equal foo # stdout omitted
```

`path` / `file` / `directory` **subject**

```
Path data-file=/tmp/data.txt
The path data-file should exist
```

`function` **subject**

```
The result of function foo should be successful
The result of "foo()" should be successful # shorthand
```

`value` **subject**

```
The value "foo" should equal "foo"
```

I do not recommend using this subject as it may not generate clear failure messages. Use the `variable` subject instead.

`variable` **subject**

```
The variable var should equal "foo"
```

Modifiers

Modifier	Description
line	The specified line of the subject.
lines	The number of lines of the subject.
word	The specified word of the subject.
length	The length of the subject.
contents	The contents of the file as subject.
result	The result of the function as subject.

`line` **modifier**

```
The line 1 of stdout should equal "line1"
```

`lines` **modifier**

```
The lines of stdout should equal 5
```

`word` **modifier**

```
The word 2 of stdout should equal "word2"
```

`length` modifier

```
The length of value "abcd" should equal 5
```

`contents` modifier

```
The contents of file "/tmp/file.txt" should equal "temp data"
```

`result` modifier

```
get_version() {
  # The result of the evaluation is passed as arguments
  # $1: stdout, $2: stderr, $3: status
  echo "$1" | grep -o '[0-9.]*' | head -1
}

When call echo "GNU bash, version 4.4.20(1)-release (x86_64-pc-linux-gnu)"
The result of function get_version should equal "4.4.20"
The result of "get_version()" should equal "4.4.20" # shorthand
```

```
check_version() {
  # The result of the evaluation is passed as arguments
  # $1: stdout, $2: stderr, $3: status
  [ "$("$1" | grep -o '[0-9.]*' | head -1)" = "4.4.20" ]
}

When call echo "GNU bash, version 4.4.20(1)-release (x86_64-pc-linux-gnu)"
The result of function check_version should be successful
The result of "check_version()" should be successful # shorthand
```

Matchers

`satisfy` matcher

Matcher	Description
satisfy	The subject should satisfy `<FUNCTION>`

```
satisfy <FUNCTION> [ARGUMENTS...]
```

satisfy examples

```
value() {
  # The subject is stored in the same variable name as the function name
  test "${value:?}" "$1" "$2"
}

formula() {
  value=${formula:?}
  [ $(($1)) -eq 1 ]
}

When call echo "50"
The output should satisfy value -gt 10
The output should satisfy formula "10 <= value && value <= 100"
```

stat matchers

the subject expected file path

Matcher	Description
exist	The file should exist.
~~be exist~~	The file should exist. (deprecated)
be file	The file should be a file.
be directory	The file should be a directory.
be empty file	The file should be an empty file.
be empty directory	The directory should be an empty directory.
be symlink	The file should be a symlink.
be pipe	The file should be a pipe.
be socket	The file should be a socket.
be readable	The file should be readable.
be writable	The file should be writable.
be executable	The file should be executable.
be block_device	The file should be a block device.
be character_device	The file should be a character device.
~~has setgid~~	The file should have the setgid flag set. (deprecated)
have setgid	The file should have the setgid flag set.
~~has setuid~~	The file should have the setuid flag set. (deprecated)
have setuid	The file should have the setuid flag set.

`exist` **matcher**

```
The path /target/path should exist
```

or

```
The path /target/path should be exist
```

`be file` **matcher**

```
The path /target/path should be file
```

`be directory` **matcher**

```
The path /target/path should be directory
The path /target/path should be dir
```

`be empty file` **matcher**

```
The path /target/path should be empty file
```

`be empty directory` **matcher**

```
The path /target/path should be empty directory
The path /target/path should be empty dir
```

`be symlink` **matcher**

```
The path /target/path should be symlink
```

`be pipe` **matcher**

```
The path /target/path should be pipe
```

`be socket` **matcher**

```
The path /target/path should be socket
```

`be readable` **matcher**

```
The path /target/path should be readable
```

`be writable` **matcher**

```
The path /target/path should be writable
```

`be executable` **matcher**

```
The path /target/path should be executable
```

`be block_device` **matcher**

```
The path /target/path should be block_device
```

`be character_device` **matcher**

```
The path /target/path should be character_device
```

`have setgid` **matcher**

```
The path /target/path should have setgid
```

`have setuid` **matcher**

```
The path /target/path should have setuid
```

status matchers

the subject expected status

Matcher	Description
be success	The status should be success (`0`).
be failure	The status should be failure (`1` - `255`).

`be success` **matcher**

```
The status should be success
```

`be failure` **matcher**

```
The status should be failure
```

string matchers

Matcher	Description
equal `<STRING>` eq `<STRING>`	The subject should equal `<STRING>`
start with `<STRING>`	The subject should start with `<STRING>`
end with `<STRING>`	The subject should end with `<STRING>`
include `<STRING>`	The subject should include `<STRING>`
match pattern `<PATTERN>`	The subject should match pattern `<PATTERN>`

`equal` **matcher**

```
The output should equal <STRING>
The output should eq <STRING>
```

`start with` **matcher**

```
The output should start with <STRING>
```

`end with` **matcher**

```
The output should end with <STRING>
```

`include` **matcher**

```
The output should include <STRING>
```

`match pattern` **matcher**

```
The output should match pattern <PATTERN>
```

PATTERN examples

- `foo*`

- `foo?`
- `[fF]oo`
- `[!F]oo`
- `[a-z]`
- `foo|bar`

`successful` matcher

Use with result modifier.

valid matchers

Plan to deprecate in the future.

Matcher	Description
be valid number	The subject should be a valid number.
be valid funcname	The subject should be a valid funcname.

variable matchers

the subject expect variable

Matcher	Description
be defined	The variable should be defined (set).
be undefined	The variable should be undefined (unset).
be present	The variable should be present (non-zero length string).
be blank	The variable should be blank (unset or zero length string).
be exported	The variable should be exported.
be readonly	The variable should be readonly.

`be defined` matcher

```
The variable VAR should be defined
```

`be undefined` matcher

```
The variable VAR should be undefined
```

`be present` matcher

```
The variable VAR should be present
```

`be blank` matcher

```
The variable VAR should be blank
```

`be exported` **matcher**

```
The variable VAR should be exported
```

`be readonly` **matcher**

```
The variable VAR should be readonly
```

Helper

Skip / Pending

DSL	Description
Skip `<REASON>`	Skip current block.
Skip if `<REASON>` `<FUNCTION> [ARGUMENTS...]`	Skip current block with conditional.
Pending `<REASON>`	Pending current block.
Todo	Define pending example

`Skip`

`Skip if`

`Pending`

`Todo`

Data

DSL	Description
Data[:raw] #|... End	Define stdin data for evaluation (without expand variables).
Data:expand #|... End	Define stdin data for evaluation (with expand variables).
Data `<FUNCTION> [ARGUMENTS...]`	Use function for stdin data for evaluation.
Data `"<STRING>"` Data `'<STRING>'`	Use string for stdin data for evaluation.
Data `< <FILE>`	Use file for stdin data for evaluation.

NOTE: The `Data` helper can also be used with filters.

```
Data | tr 'abc' 'ABC' # comment
#|aaa
#|bbb
#|ccc
End
```

`Data[:raw]`

```
Describe 'Data helper'
  Example 'provide with Data helper block style'
    Data
      #|item1 123
      #|item2 456
      #|item3 789
    End
    When call awk '{total+=$2} END{print total}'
    The output should eq 1368
  End
End
```

`Data:expand`

`Data <FUNCTION>`

`Data "<STRING>"`

`Data < "<FILE>"`

Parameters

DSL	Description
Parameters ... End	Define parameters (block style)
Parameters:block ... End	Same as Parameters
Parameters:value `[VALUES...]`	Define parameters (value style)
Parameters:matrix ... End	Define parameters (matrix style)
Parameters:dynamic ... End	Define parameters (dynamic style)

NOTE: Multiple `Parameters` definitions are merged.

`Parameters[:block]`

```
Describe 'example'
  Parameters
    "#1" 1 2 3
    "#2" 1 2 3
  End

  Example "example $1"
    When call echo "$(($2 + $3))"
    The output should eq "$4"
  End
End
```

`Parameters:value`

```
Parameters:value foo bar baz
```

`Parameters:matrix`

```
Parameters:matrix
  foo bar
  1 2

  # expanded as follows
  #   foo 1
  #   foo 2
  #   bar 1
  #   bar 2
End
```

`Parameters:dynamic`

```
Parameters:dynamic
  for i in 1 2 3; do
    %data "#$i" 1 2 3
  done
End
```

Only `%data` directives can be used within a `Parameters:dynamic` block. You can not call a function or access variables defined within the specfile. You can refer to variables defined with `%const`.

Others

DSL	Description
Include `<NAME>`	Include other files.
Path File Dir	Define a path alias.
Intercept `[NAMES...]`	Define an interceptor.
Set `[OPTION:<on | off>...]`	Set shell option before running each example.
Dump	Dump stdout, stderr and status for debugging.

`Include`

`Path` / `File` / `Dir`

`Intercept`

`Set`

`Dump`

Hooks

DSL	Description
Before	Define a hook called before running each example.
After	Define a hook called after running each example.
BeforeAll	
AfterAll	
BeforeCall	
AfterCall	
BeforeRun	
AfterRun	

`Before` / `After`

`BeforeAll` / `AfterAll`

`BeforeCall` / `AfterCall`

`BeforeRun` / `AfterRun`

Directive

Directive	Description
%const, %	Define a constant variable.
%text	Define a multiline texts to output to stdout.
%putsn, %=	Output arguments with the newline.
%puts, %-	Output arguments.
%logger	Output log message.

`%const` (`%`)

```
% <VERNAME>: "<VALUE>"
```

`%text`

`%puts` (`%-`) / `%putsn` (`%=`)

%preserve

Use this with the `When run` evaluation.

`%logger`

Special environment Variables

ShellSpec provides special environment variables with prefix `SHELLSPEC_`. They are useful for writing tests and extensions. I'll try to avoid making breaking changes to these, but can't guarantee it. There are many undocumented variables. You can use them at your own risk.

These variables can be overridden by the `--env-from` option, except for some variables. This is an assumed usage, but has not been fully tested. It is recommended to use it as read-only.

Name	Description	Value
SHELLSPEC_ROOT	ShellSpec root directory	
SHELLSPEC_LIB	ShellSpec lib directory	`${SHELLSPEC_ROOT}/lib`
SHELLSPEC_LIBEXEC	ShellSpec libexec directory	`${SHELLSPEC_ROOT}/libexec`
SHELLSPEC_TMPDIR	Temporary directory	`${TMPDIR}` or `/tmp` if not specified.
SHELLSPEC_TMPBASE	Temporary directory used by ShellSpec	`${SHELLSPEC_TMPDIR}/shellspec.${SHELLSPEC_UNIXTIME}.$$`.
SHELLSPEC_WORKDIR	Temporary directory for each spec number	`${SHELLSPEC_TMPBASE}/${SHELLSPEC_SPEC_NO}`.

Name	Description	Value
SHELLSPEC_PROJECT_ROOT	Where `.shellspec` is located	
~~SHELLSPEC_SPECDIR~~	Specfiles directory	`${SHELLSPEC_PROJECT_ROOT}/spec` [depricated]
SHELLSPEC_HELPERDIR	spec_helper directory	`${SHELLSPEC_PROJECT_ROOT}/spec` (default)
SHELLSPEC_LOAD_PATH	Load path of library	`${SHELLSPEC_SPECDIR}:${SHELLSPEC_LIB}:${SHELLSPEC_LIB}/formatters`
SHELLSPEC_UNIXTIME	Unix Time when ShellSpec starts	
SHELLSPEC_SPECFILE	Current running specfile path	
SHELLSPEC_SPEC_NO	Current specfile number	
SHELLSPEC_GROUP_ID	Current group ID	e.g. `1-2`
SHELLSPEC_EXAMPLE_ID	Current example ID (including group ID)	e.g. `1-2-3`
SHELLSPEC_EXAMPLE_NO	Current serial number of example	

Go to TOC

Reporter

Reporting protocol

Format

```
<RS>key1:value1<US>key2:value2<US>key3:value3<US>...<ETB>
<RS>key1:value1<US>key2:value2<US>key3:value3<US>...<ETB>
<RS>key1:value1<US>key2:value2<US>key3:value3<US>...<ETB>
...
```

- The value may contain newlines.
- Ignore data that is not enclosed in `RS` and `ETB`.
  - Note: The reporter will output the ignored data as is.
- The `RS`, `US`, and `ETB` in the value must be removed.

Structure

pair	type	number of times
	meta	1
+-----	begin	0..N (number of specfiles)
: +--	example	0..N
: :	statement	0..N
: +--	result	same number of times as example
+-----	end	same number of times as begin
	finished	1
	error	0..N (can occur at any time)

types

type	fields
meta	shell, shell_type, shell_version, info
begin	specfile
example	id, block_no, example_no, focused, description, lineno_range, stdout, stderr
statement	tag, lineno, [statement type fields...]
result	[result type fields...], trace
end	example_count
finished	

type	fields
error	lineno, note, message, failure_message

statement type

tag	fields
evaluation	note, fail:, evaluation
good	note, fail:, message
bad	note, fail:y, message, failure_message
warn	note, fail:?, message, failure_message
skip	note, fail:, message, skipid, temporary:?
pending	note, fail:, message, pending, temporary:?

result type

tag	fields
succeeded	note, fail: , quick:
failed	note, fail:y, quick:y
warned	note, fail:?, quick:?
todo	note, fail: , quick:?, temporary:?,
fixed	note, fail:y, quick:?, temporary:?,
skipped	note, fail: , quick: , temporary:?,

- warned
  - fail: 'y' if enabled warning-as-failure, otherwise empty
  - quick: 'y' if enabled warning-as-failure, otherwise empty
- todo
  - temporary: 'y' if pending statement is temporary, otherwise empty
  - quick: 'y' if it is temporary
- fixed
  - temporary: 'y' if pending statement is temporary, otherwise empty
  - quick: 'y' if it is temporary
- skipped
  - temporary: 'y' if pending statement is temporary, otherwise empty

Go to TOC

Shells

- Tested shells and versions
  - Packages
    - Alpine / BusyBox / OpenWrt (LEDE)
    - AlmaLinux / Rocky Linux / CentOS / Fedora
    - Debian / Ubuntu
    - FreeBSD
    - macOS
    - Windows
  - Manual test
    - Solaris
    - OpenBSD / NetBSD
    - AIX
  - Experimental
    - Schily Bourne Shell
    - GWSH shell
- Confirmation for bug
- Built-in commands

Tested shells and versions

- The shell included with the supported OS (the platform in bold) is the main supported shell.
- Support for very old shells may be discontinued without notice.
- The version with a dash does not work.
- The version in italic may work, but some features are not working or unstable due to bugs in the shell.
- Bourne shell (pre-POSIX shell) is not supported.

Packages

- These are tested by GitHub Actions, Travis CI, Cirrus CI and Docker (`contrib/test_in_docker.sh`).

Alpine / BusyBox / OpenWrt (LEDE)

Default shell: `busybox ash`

Platform	bash	busybox	dash	loksh	mksh	oksh	yash	zsh
Alpine 3.15.4	5.1.16	1.34.1		7.0			-	
Alpine edge				7.1		7.0	-	
BusyBox 1.34.1	-	1.34.1	-	-	-		-	-
LEDE 17.01.7		1.25.1	-	-			-	
OpenWrt 10.03.1		*1.15.3*		-			-	-

Platform	bash	busybox	dash	loksh	mksh	oksh	yash	zsh
OpenWrt 12.09		*1.19.4*		-			-	-
OpenWrt 14.07		1.22.1		-			-	
OpenWrt 15.05.1		1.23.2	-	-			-	
OpenWrt 18.06.9		1.28.4	-	-			-	
OpenWrt 19.07.9		1.30.1	-	-			-	
OpenWrt 21.02.3		1.33.2	-	-			-	

- *busybox 1.15.3, 1.19.4*: `readonly` is not working properly.
- `busybox hush` not supported.

AlmaLinux / Rocky Linux / CentOS / Fedora

Default shell: `bash`

Platform	bash	busybox	dash	ksh	mksh	yash	zsh
AlmaLinux 8.5	4.4.20	-	-			-	
Rocky Linux 8.5	4.4.20	-	-			-	
CentOS 7.9.2009	4.2.46	-	-			-	
CentOS 8.4.2105	4.4.19	-	-			-	5.5.1
fedora 34	5.0.11						
fedora 34	5.0.17						

- Testing bash 3.2.25- in POSIX mode only.

Debian / Ubuntu

Default shell: `dash` or `bash` (until debian 5.0)

Platform	bash	busy-box	dash	ksh93	ksh2020	mksh	pd-ksh	posh	yash	zsh
Debian 2.2r7	2.03	-	-	-	-	-	*5.2.14*	-	-	3.1.9
Debian 3.0r6	2.05a	~~0.60.2~~	~~ash 0.3.8~~	-	-	-	5.2.14	-	-	4.0.4
Debian 3.1r8	2.05b	~~0.60.5~~	0.5.2	*93q*	-	-	5.2.14	0.3.14	-	*4.2.5*
Debian 4.0r9	3.1.17	*1.1.3*	*0.5.3*	*93r*	-	R28	5.2.14	0.5.4	-	4.3.2

Platform	bash	busy-box	dash	ksh93	ksh2020	mksh	pd-ksh	posh	yash	zsh
Debian 5.0.10	3.2.39	*1.10.2*	0.5.4	93s+ 2008-01-31	-	R35	5.2.14	0.6.13	-	4.3.6
Debian 6.0.10	4.1.5	1.17.1	0.5.5.1	93s+ 2008-01-31	-	R39	5.2.14	*0.8.5*	-	4.3.10
Debian 7.11	4.2.37	1.20.0	0.5.7	93u+ 2012-02-29	-	R40	-	*0.10.2*	2.30	4.3.17
Debian 8.11	4.3.30	1.22.0	0.5.7	93u+ 2012-08-01	-	R50d	-	0.12.3	2.36	5.0.7
Debian 9.13	4.4.12	1.22.0	0.5.8	93u+ 2012-08-01	-	R54	-	0.12.6	2.43	5.3.1
Debian 10.12	5.0.3	1.30.1	0.5.10.2	93u+ 2012-08-01	-	R57	-	0.13.2	2.48	5.7.1
Debian 11.3	5.1.14	1.30.1	0.5.11	93u+ 2012-08-01	-	R59	-	0.14.1	2.50	5.8
Debian Bookworm	5.1.16			93u+m 1.0.0 beta.2	-		-		2.52	5.8.1
Ubuntu 12.04	4.2.25	1.18.5	0.5.7	93u 2011-02-08	-	R40	-	*0.10*	2.29	4.3.17
Ubuntu 14.04	4.3.11	1.21.0	0.5.7	93u+ 2012-08-01	-	R46	-	0.12.3	2.35	5.0.2
Ubuntu 16.04	4.3.48	1.22.0	0.5.8	93u+ 2012-08-01	-	R52c	-	0.12.6	2.39	5.1.1
Ubuntu 18.04	4.4.20	1.27.2	0.5.8	93u+ 2012-08-01	-	R56c	-	0.13.1	2.46	5.4.2

Platform	bash	busy-box	dash	ksh93	ksh2020	mksh	pd-ksh	posh	yash	zsh
Ubuntu 20.04	5.0.16	1.30.1	0.5.10.2	93u+ 2012-08-01	2020.0.0	R58	-	0.13.2	2.49	5.8

- Use debian/eol docker images for older Debian versions.
- Testing bash 2.03-3.2.39 in both POSIX and non-POSIX mode. bash 4.1.5- is non-POSIX mode only.
- *pdksh 5.2.14* (Debian 2.2.r7): `readonly` is not working properly.
- *ksh 93q, 93r*: Sometimes abort due to unstable bugs.
- *busybox 1.1.3*: Builtin commands cannot be mocked.
- *busybox 1.10.2*: It may not work properly due to a bug.
- *posh 0.8.5*: Signal handling (`trap`) broken.
- *posh 0.10, 0.10.2*: Shell flags (e.g. `set -e`, `set -u`) not working properly.
- *zsh 4.2.5*: errexit handling broken.

FreeBSD

Default shell: `ash`

Platform	FreeBSD sh	bash	dash	ksh	mksh	oksh	pdksh	zsh
FreeBSD 11.4	unknown	5.1.8	0.5.11.4	93u+ 2012-08-01	R59c	6.9	*5.2.14*	5.8
FreeBSD 12.3	unknown	5.1.16	0.5.11.5	93u+ 2012-08-01	R59c	7.0	*5.2.14*	5.8.1
FreeBSD 13.0	unknown	5.1.16	0.5.11.5	93u+ 2012-08-01	R59c	7.0	*5.2.14*	5.8.1

- *pdksh 5.2.14*: Unstable on CI environment (Bus Error, Bad address, Memory fault, core dumped and etc).

macOS

Default shell: `bash`

Platform	sh	bash	dash	ksh	mksh	posh	yash	zsh
macOS 10.10	bash 3.2.57	3.2.57	-	93u+ 2012-08-01	-	-	-	5.0.5
macOS 10.11	bash 3.2.57	3.2.57	-	93u+ 2012-08-01	-	-	-	5.0.8
macOS 10.12	bash 3.2.57	3.2.57	-	93u+ 2012-08-01	-	-	-	5.2
macOS 10.13	bash 3.2.57	3.2.57	-	93u+ 2012-08-01	-	-	-	5.3

Platform	sh	bash	dash	ksh	mksh	posh	yash	zsh
macOS 10.14.4 (Mojave)	bash 3.2.57	3.2.57	-	93u+ 2012-08-01	-	-	-	5.3
macOS 10.15.5 (Catalina)	bash 3.2.57	3.2.57	-	93u+ 2012-08-01	-	-	-	5.7.1
macOS 11.6.2 (Big Sur)	bash 3.2.57	3.2.57	-	93u+ 2012-08-01	-	-	-	5.8
macOS 12.3.1 (Monterey)	bash 3.2.57	3.2.57	-	93u+ 2012-08-01	-	-	-	5.8
macOS 12.3.1 (brew)	-	5.1.0	0.5.11.2	2020.0.0	R59b	~~0.14.1~~	2.50	5.8

- Supports the latest three versions.
- Homebrew version of posh is broken.

Windows

Default shell: `bash`

Platform	bash	busybox	dash	mksh	posh	zsh
Windows Server 2019 (Git Bash)	4.4.23	-	unknown	-	-	-
Windows Server 2019 (msys)	5.1.16	1.31.1	0.5.11.5	R59	-	5.8
Windows Server 2019 (cygwin)	4.4.12	1.23.2	0.5.11.5	R56c	0.13.2	5.8
Windows Server 2019 (busybox-w32)	-	1.35.0 (4487)	-	-	-	-

- busybox-w32: https://frippery.org/busybox/
  - Versions that do not work properly
    - 4264, 4621

Manual test

This is not continuous test, it may break sometimes...

- * Reported by user

Solaris

Platform	/bin/sh	/usr/bin/ksh	/usr/sunos/bin/ksh	/usr/sunos/bin/sh
Solaris 10	~~Bourne Shell~~	*ksh88 M-11/16/88i*	-	-
Solaris 11	ksh93 93u+ 2012-08-01	-	*ksh88 M-11/16/88i*	~~Bourne Shell~~

- *ksh88 M-11/16/88i*: Builtin commands cannot be mocked.

OpenBSD / NetBSD

Platform	/bin/sh	/usr/bin/ksh
FreeBSD 13 current	FreeBSD sh	-
OpenBSD 6.6	OpenBSD ksh (POSIX mode pdksh) 5.2.14	pdksh 5.2.14
NetBSD 9.0	NetBSD sh 20181212 BUILD:20200214000628Z	pdksh 5.2.14

AIX

Platform	Shells
AIX 7.2.1.4 *	bash 4.3.30

Experimental

These are tested by Docker (`contrib/test_in_docker.sh`).

Schily Bourne Shell

Platform	Schily versions	bosh / pbosh
Debian bullseye	AN-2021-09-18	2021/07/23

- Schily Bourne Shell (`bosh` , `pbosh`)
- Packages are available on The NetBSD package collection.
- Versions before 2018/10/07 not tested (probably not working).

GWSH shell

Platform	gwsh
Debian bullseye	snapshot-20190627

- GWSH shell

Confirmation for bug

`contrib/bugs.sh` detects shell bugs and problems.

Usage: `contrib/bugs.sh`

Built-in commands

`contrib/builtins.sh` is a script for listing built-in commands.

Usage: `contrib/builtins.sh`

Go to TOC

How to test

contrib/all.sh

Use `contrib/all.sh` to test on all installed shells.

Usage: `contrib/all.sh [COMMAND (shellspec and etc)]`

contrib/test_in_docker.sh

Use `contrib/test_in_docker.sh` to test on supported shells (requires Docker).

Usage: `contrib/test_in_docker.sh <DOCKERFILES...> [-- COMMAND]`

The Dockerfiles are in the `dockerfiles` directory (the files whose name begin with `.` are for unsupported shells).

contrib/check.sh

Use `contrib/check.sh` to check syntax on the whole project (requires docker).

Usage: `contrib/check.sh`

contrib/installer_test.sh

`contrib/installer_test.sh` is useful for creating a test environment for the installer (requires docker).

Usage: `contrib/installer_test.sh`

Go to TOC

Colophon

This book is created by using the following sources:

- Shellspec - English
- GitHub source: shellspec/shellspec/docs
- Created: 2022-11-27
- Bash v5.2.2
- Vivliostyle, https://vivliostyle.org/
- By: @shinokada
- GitHub repo: https://github.com/shinokada/markdown-docs-as-pdf